

*Projets IoT | ESP32 + Wokwi + Blynk
Master GLCC Informatique - FS Ibn Tofail Kenitra*

RAPPORT DE MINI-PROJET

Projet 5

Surveillance de la qualité de l'air intérieur

Microcontrôleur	ESP32 DevKit C v4
Simulateur	Wokwi (composants natifs)
Langage	Arduino C/C++
Supervision IoT	Blynk (Wi-Fi Wokwi-GUEST)
Lien Wokwi	wokwi.com/projects/465825450297283585

Realise par :

Mohamed El Haimer
El Ouazzani Mehdi Rayane
Ilias Jebrane

Annee universitaire 2025 - 2026

Sommaire

1. Introduction et contexte

2. Analyse du besoin (Partie A)

2.1 Indicateurs de qualité de l'air

2.2 Indice de confort global

2.3 Niveaux de qualité

2.4 Schéma fonctionnel

3. Liste des composants (BOM) et justification

4. Cablage et affectation des broches

5. Architecture logicielle

5.1 Gestion des cas d'erreur

6. Code source et commentaires (sketch.ino)

7. Intégration Blynk

7.1 Datastreams (Virtual Pins)

8. Validation et tableau de tests

9. Extensions réalisées

9.1 Vers une version réelle

10. Difficultés rencontrées

11. Conclusion

12. Annexes - Liens

1. Introduction et contexte

Ce projet a pour objectif de concevoir un système connecté de surveillance de la qualité de l'air dans une pièce d'habitation (salon, chambre). Le système mesure ou simule trois grandeurs : la température, l'humidité relative et un indice de pollution (CO₂ / COV). À partir de ces grandeurs, il calcule un indice de confort global et classe l'air en trois niveaux : sain, modéré ou pollué.

Les résultats sont affichés localement sur un écran OLED (deux écrans qui alternent automatiquement) et transmis à distance vers une application Blynk via le Wi-Fi. Des LEDs et un buzzer assurent une signalisation locale immédiate, et des notifications push sont envoyées à l'utilisateur lorsque l'air devient pollué.

La réalisation est entièrement simulée sur Wokwi avec un ESP32 DevKit C v4, en langage Arduino C/C++, conformément au cahier des charges.

2. Analyse du besoin

2.1 Indicateurs de la qualité de l'air

La qualité de l'air intérieur dépend principalement de trois indicateurs :

- Température : le confort thermique se situe entre 18 et 24°C.
- Humidité relative : zone de confort entre 40 % et 60 % -> trop basse, l'air est sec ; trop haute, risque de moisissures.
- CO₂ / COV : un taux élevé traduit un air vicié et impose une aération. Il est simulé par un potentiomètre (0 à 100 %).

2.2 Indice de confort global

Chaque grandeur est convertie en un score de 0 à 100, puis combinée par une moyenne pondérée :

$$\text{Indice global} = T(30 \%) + HR(30 \%) + CO_2(40 \%)$$

Le CO₂ reçoit le poids le plus élevé (40 %) car il est l'indicateur le plus critique pour la salubrité de l'air.

2.3 Niveaux de qualité

Niveau	Indice global	LED	Buzzer
Sain	Indice ≥ 60	Verte	Eteint
Modéré	$30 \leq \text{Indice} < 60$	Orange	Eteint
Pollué	Indice < 30	Rouge	Bip intermittent

2.4 Schéma fonctionnel

Le flux de traitement suit la chaîne : capteurs -> ESP32 (traitement et décision) -> sorties locales (OLED, LEDs, buzzer) et communication Blynk.

DHT22 + Potentiomètre + NTC -> ESP32 -> OLED + LEDs + Buzzer + Blynk (Wi-Fi)

Schema fonctionnel du systeme

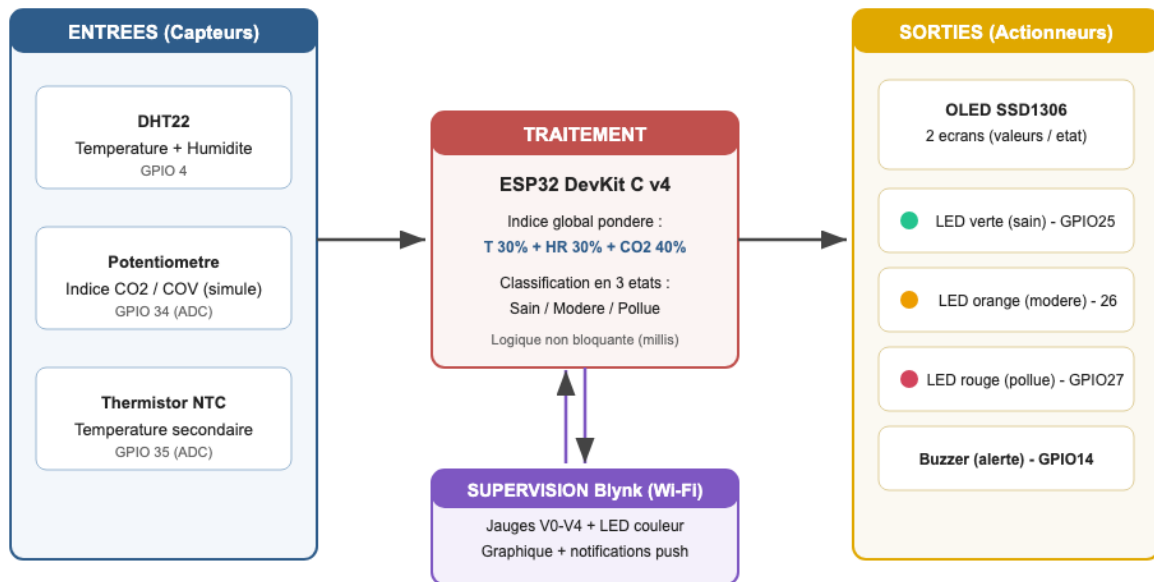


Figure 1 : Schema fonctionnel (entrees / traitement / sorties / Blynk)

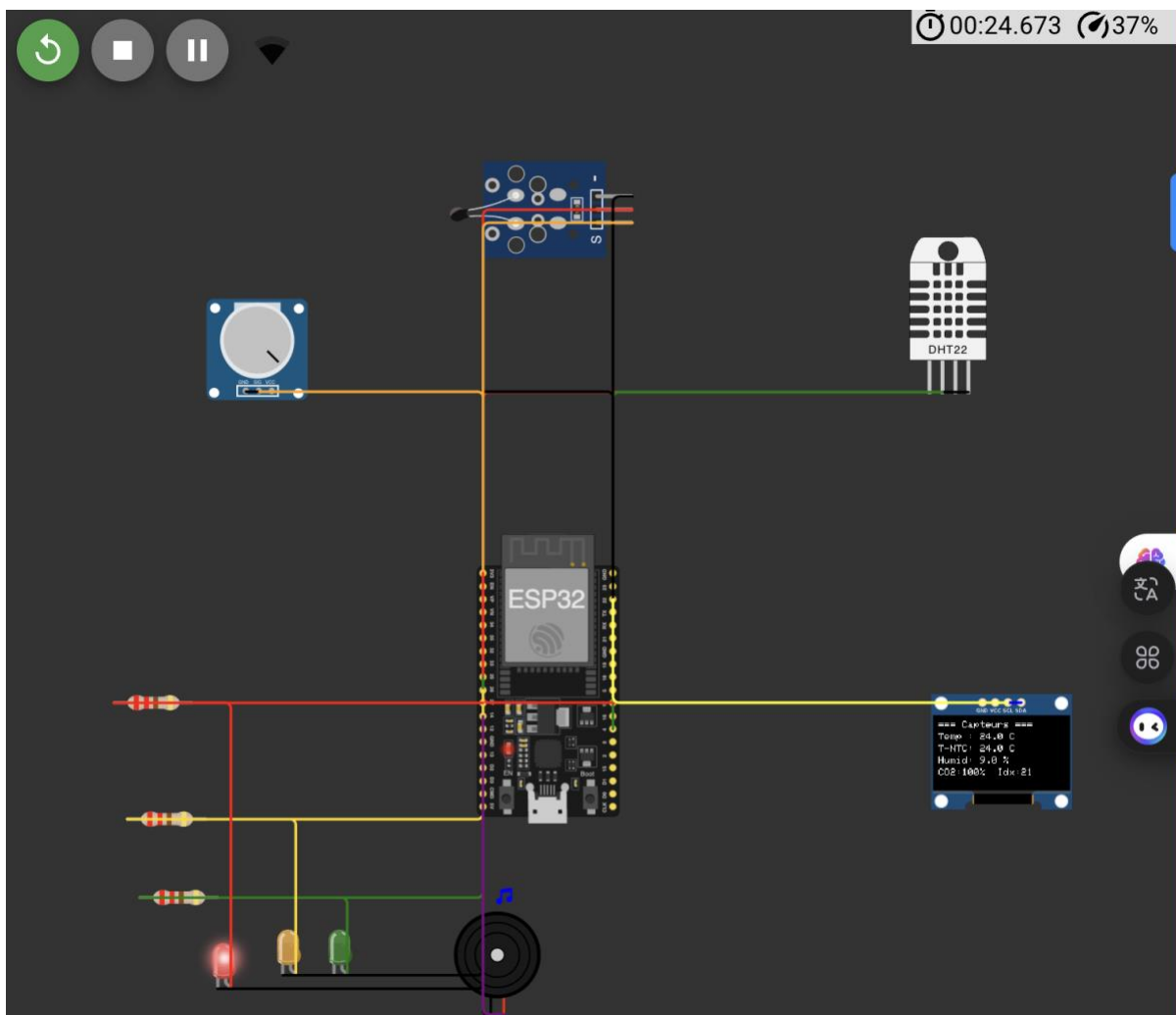
3. Liste des composants (BOM) et justification

Composant	Rôle dans le projet	Justification du choix
ESP32 DevKit C v4	Unité de traitement + Wi-Fi	Microcontrôleur imposé, Wi-Fi intégré pour Blynk
DHT22	Température + humidité	Capteur natif Wokwi, mesure les 2 grandeurs clés
Potentiomètre	Indice CO2/COV simulé	Simule un capteur analogique absent de Wokwi
NTC thermistor	Température secondaire	Contrôle croisé de la température (redondance)
OLED SSD1306 I2C	Afficheur principal	Affichage multi-écrans lisible, bus I2C
LED verte/orange/rouge	Signalisation des 3 états	Indication visuelle immédiate du niveau
Buzzer	Alerte sonore critique	Alerte active uniquement en état pollué
3 résistances 220 Ohm	Protection des LEDs	Limitation du courant dans les LEDs

4. Cablage et affectation des broches

Le tableau ci-dessous résume les broches utilisées, conformes au cahier des charges.

Composant	Broche ESP32	Type
DHT22 (data)	GPIO 4	Numerique (OneWire)
Potentiometre (CO2)	GPIO 34	Analogique (ADC)
NTC thermistor	GPIO 35	Analogique (ADC)
LED verte (sain)	GPIO 25	Numerique (sortie)
LED orange (modere)	GPIO 26	Numerique (sortie)
LED rouge (pollue)	GPIO 27	Numerique (sortie)
Buzzer	GPIO 14	Numerique (sortie)
OLED SDA / SCL	GPIO 21 / GPIO 22	Bus I2C



5. Architecture logicielle

Le code est structuré en fonctions distinctes, sans logique dans `loop()`, selon le flux Entrées -> Traitement -> Sorties -> Communication. Toutes les temporisations utilisent `millis()` (aucun `delay()` dans la boucle principale).

Fonction	Role
<code>setup()</code>	Initialise capteurs, OLED, broches et connexion Blynk
<code>readNTC()</code>	Lit le thermistor NTC et le convertit en degrés Celsius
<code>readSensors()</code>	Lit DHT22, NTC et potentiomètre ; gère les valeurs NaN
<code>calculateIndex()</code>	Calcule l'indice global pondéré et l'état (0/1/2)
<code>updateLEDs()</code>	Allume la LED correspondante a l'état courant
<code>updateBuzzer()</code>	Bip intermittent uniquement en état critique
<code>getAdvice()</code>	Retourne un conseil contextuel (aération, humidificateur)
<code>showScreen1 / showScreen2</code>	Affichent les 2 écrans OLED (valeurs / états)
<code>updateDisplay()</code>	Alterne les 2 écrans toutes les 3 secondes
<code>sendToBlynk()</code>	Envoie les mesures vers V0-V4 et la couleur d'état
<code>handleNotifications()</code>	Alertes Blynk (entrée polluée durant plus de 30 min)
<code>loop()</code>	Boucle non bloquante : <code>Blynk.run()</code> + traitements <code>millis()</code>

5.1 Gestion des cas d'erreur

- Valeur hors plage / capteur DHT muet : les lectures NaN sont ignorées, la dernière valeur valide est conservée.
- Protection contre la division par zéro durant le calcul de la température NTC.
- Perte Wi-Fi : l'envoi vers Blynk est conditionné par `Blynk.connected()`.

6. Code source et commentaires (sketch.ino)

Code complet, structure en fonctions et commentaires (rôles, paramètres, valeurs de retour) :

```

/* =====
PROJET 5 - Surveillance de qualite de l'air interieur
ESP32 DevKit C v4 + Wokwi + Blynk (Wi-Fi simule Wokwi-GUEST)
Capteurs : DHT22, Potentiometre (CO2 simule), NTC (T secondaire)
Sorties : OLED SSD1306, 3 LEDs (vert/orange/rouge), Buzzer
===== */

#define BLYNK_TEMPLATE_ID "TMPL2DFMOQumD"
#define BLYNK_TEMPLATE_NAME "IOT"
#define BLYNK_AUTH_TOKEN "*****" // ne pas publier
#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_GFX.h>
#include <Wire.h>
#include <math.h>

// ===== PIN DEFINITIONS =====
#define PIN_DHT 4
#define PIN_POT 34
#define PIN_NTC 35
#define PIN_LED_GREEN 25 // air sain
#define PIN_LED_ORANGE 26 // qualite moderee
#define PIN_LED_RED 27 // air pollue
#define PIN_BUZZER 14 // alerte critique

#define DHT_TYPE DHT22
DHT dht(PIN_DHT, DHT_TYPE);

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

char ssid[] = "Wokwi-GUEST";
char pass[] = "";
const float NTC_BETA = 3950.0;

float temperature = 22.0;
float tempNTC = 22.0;
float humidity = 50.0;
int co2Index = 0;
float globalIndex = 0;
int airQuality = 0; // 0=sain 1=modere 2=pollue
int lastAirQuality = 0;

unsigned long lastSensorRead = 0;
unsigned long lastScreenSwitch = 0;
unsigned long lastBuzzerToggle = 0;
unsigned long lastBlynkSend = 0;
unsigned long pollueStartTime = 0;
bool pollueNotified30 = false;
int currentScreen = 0;
int lastScreen = -1;
bool buzzerState = false;
unsigned long stateSeconds[3] = {0, 0, 0};

// setup : initialise capteurs, OLED, Wi-Fi/Blynk, broches.
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED_GREEN, OUTPUT);
  pinMode(PIN_LED_ORANGE, OUTPUT);
  pinMode(PIN_LED_RED, OUTPUT);

```



```

pinMode(PIN_BUZZER,      OUTPUT);

// Test des sorties (delay autorise uniquement dans setup)
digitalWrite(PIN_LED_GREEN, HIGH); delay(300); digitalWrite(PIN_LED_GREEN, LOW);
digitalWrite(PIN_LED_ORANGE, HIGH); delay(300); digitalWrite(PIN_LED_ORANGE, LOW);
digitalWrite(PIN_LED_RED, HIGH); delay(300); digitalWrite(PIN_LED_RED, LOW);
digitalWrite(PIN_BUZZER, HIGH); delay(200); digitalWrite(PIN_BUZZER, LOW);

dht.begin();
Wire.begin(21, 22);
if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { while (true); }
display.clearDisplay();
display.setTextSize(1); display.setTextColor(SSD1306_WHITE);
display.setCursor(10, 20); display.println("Air Quality Monitor");
display.setCursor(30, 40); display.println("Connexion WiFi...");
display.display();
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}

// readNTC : lit le NTC et convertit en degres C. retour : float
float readNTC() {
  int raw = analogRead(PIN_NTC);
  if (raw <= 0) raw = 1;
  if (raw >= 4095) raw = 4094;
  float celsius = 1.0 / (log(1.0 / (4095.0 / raw - 1.0)) / NTC_BETA
    + 1.0 / 298.15) - 273.15;
  return celsius;
}

// readSensors : lit DHT22, NTC, potentiometre. Gere les NaN.
void readSensors() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (!isnan(t)) temperature = t;
  if (!isnan(h)) humidity = h;
  tempNTC = readNTC();
  int rawPot = analogRead(PIN_POT);
  co2Index = map(rawPot, 0, 4095, 0, 100);
}

// calculateIndex : indice pondere T(30)+HR(30)+CO2(40) et etat.
void calculateIndex() {
  float tMoy = (temperature + tempNTC) / 2.0; // utilisation du NTC
  float tScore;
  if (tMoy < 10 || tMoy > 35) tScore = 0;
  else if (tMoy < 16 || tMoy > 28) tScore = 40;
  else if (tMoy < 18 || tMoy > 24) tScore = 70;
  else tScore = 100;
  float hScore;
  if (humidity < 20 || humidity > 80) hScore = 0;
  else if (humidity < 30 || humidity > 70) hScore = 40;
  else if (humidity < 40 || humidity > 60) hScore = 70;
  else hScore = 100;
  float co2Score = 100 - co2Index;
  globalIndex = (tScore * 0.30) + (hScore * 0.30) + (co2Score * 0.40);
  globalIndex = constrain(globalIndex, 0, 100);
  if (globalIndex >= 60) airQuality = 0;
  else if (globalIndex >= 30) airQuality = 1;
  else airQuality = 2;
}

// updateLEDs : allume la LED de l'etat courant.
void updateLEDs() {
  digitalWrite(PIN_LED_GREEN, airQuality == 0);
  digitalWrite(PIN_LED_ORANGE, airQuality == 1);
  digitalWrite(PIN_LED_RED, airQuality == 2);
}

// updateBuzzer : bip intermittent uniquement en etat pollue.
void updateBuzzer() {
  if (airQuality == 2) {

```

```

    if (millis() - lastBuzzerToggle >= 500) {
        buzzerState = !buzzerState;
        digitalWrite(PIN_BUZZER, buzzerState);
        lastBuzzerToggle = millis();
    }
} else { digitalWrite(PIN_BUZZER, LOW); buzzerState = false; }
}

// getAdvice : conseil contextuel. retour : const char*
const char* getAdvice() {
    if (co2Index > 60) return "Ouvrez fenetre";
    if (humidity < 30) return "Humidificateur";
    if (humidity > 70) return "Trop humide";
    return "Air confortable";
}

void showScreen1() {
    display.clearDisplay();
    display.setCursor(0, 0); display.println("=== Capteurs ===");
    display.setCursor(0, 12); display.printf("Temp : %.1f C", temperature);
    display.setCursor(0, 24); display.printf("T-NTC: %.1f C", tempNTC);
    display.setCursor(0, 36); display.printf("Humid: %.1f %%", humidity);
    display.setCursor(0, 48); display.printf("CO2:%d%% Idx:%.0f", co2Index, globalIndex);
    display.display();
}

void showScreen2() {
    display.clearDisplay();
    display.setCursor(0, 0); display.println("=== Etat Air ===");
    display.setTextSize(2); display.setCursor(0, 18);
    if (airQuality == 0) display.println("SAIN");
    else if (airQuality == 1) display.println("MODERE");
    else display.println("POLLUE");
    display.setTextSize(1);
    display.setCursor(0, 50); display.print("> "); display.println(getAdvice());
    display.display();
}

// updateDisplay : alterne les 2 ecrans toutes les 3 secondes.
void updateDisplay() {
    if (millis() - lastScreenSwitch >= 3000) {
        currentScreen = !currentScreen;
        lastScreenSwitch = millis(); lastScreen = -1;
    }
    if (currentScreen != lastScreen) {
        if (currentScreen == 0) showScreen1(); else showScreen2();
        lastScreen = currentScreen;
    }
}

// sendToBlynk : envoie V0-V4 + couleur d'etat. Gere Wi-Fi perdu.
void sendToBlynk() {
    if (!Blynk.connected()) return;
    Blynk.virtualWrite(V0, temperature);
    Blynk.virtualWrite(V1, humidity);
    Blynk.virtualWrite(V2, co2Index);
    Blynk.virtualWrite(V3, globalIndex);
    const char* color = (airQuality == 0) ? "#23C48E"
        : (airQuality == 1) ? "#ED9D00" : "#D3435C";
    Blynk.setProperty(V4, "color", color);
    Blynk.virtualWrite(V4, 255);
}

// handleNotifications : alerte a l'entree pollue et apres 30 min.
void handleNotifications() {
    if (airQuality == 2 && lastAirQuality != 2) {
        pollueStartTime = millis();
        pollueNotified30 = false;
        Blynk.logEvent("air_pollue", "Alerte : air interieur POLLUE !");
    }
    if (airQuality == 2 && !pollueNotified30 &&
        (millis() - pollueStartTime >= 30UL * 60UL * 1000UL)) {

```

```
    Blynk.logEvent("air_pollue", "Air pollue depuis plus de 30 minutes !");
    pollueNotified30 = true;
  }
  lastAirQuality = airQuality;
}

// loop : boucle non bloquante (Blynk.run + traitements millis).
void loop() {
  Blynk.run();
  if (millis() - lastSensorRead >= 2000) {
    readSensors(); calculateIndex(); updateLEDs(); handleNotifications();
    stateSeconds[airQuality] += 2;
    lastSensorRead = millis();
  }
  if (millis() - lastBlynkSend >= 2000) { sendToBlynk(); lastBlynkSend = millis(); }
  updateBuzzer();
  updateDisplay();
}
```

7. Intégration Blynk

7.1 Datastreams (Virtual Pins)

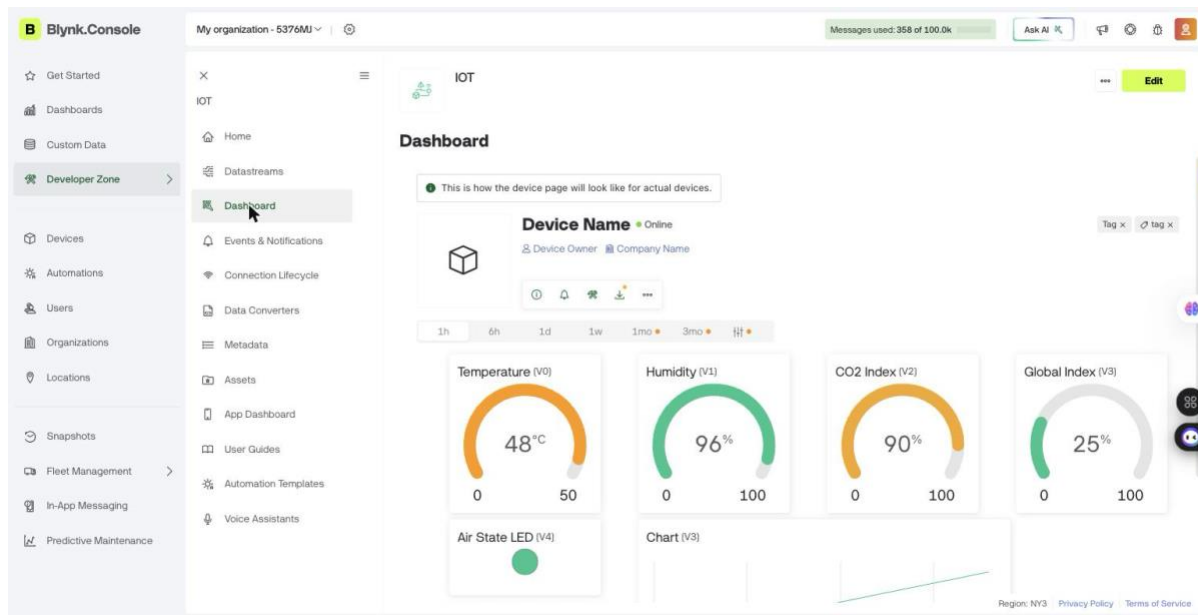
Pin	Donnee	Type	Min / Max
V0	Température	Double	0 / 50° C
V1	Humidité	Double	0 / 100 %
V2	Indice CO2 (simulé)	Integer	0 / 100 %
V3	Indice global	Double	0 / 100 %
V4	LED couleur d'état	Integer	0 / 255

Le widget LED (V4) change de couleur selon l'état : vert (sain), orange (modéré), rouge (pollué). Un graphique (Chart) sur V3 trace l'historique de l'indice global. Un événement nommé air_pollue est configuré pour les notifications push.

The screenshot displays the Blynk Console interface for an IoT project. The main section is titled 'Datastreams' and lists five virtual pins (V0-V4) with their respective names, data types, and ranges. The table is as follows:

Pin	Name	Data Type	Min	Max	Default	Units	Use in Automations
V0	Temperature	Double	0	50	25	°C	None
V1	Humidity	Double	0	100	50	%	None
V2	CO2 Index	Integer	0	100	50	%	None
V3	Global Index	Double	0	100	50	%	None
V4	Air State LED	Integer	0	255	0	-	None

The interface also includes a sidebar with navigation options, a top bar with organization information and message usage, and a bottom status bar showing the region (NY3) and links to privacy policy and terms of service.



The screenshot shows the Blynk.Console IOT Events & Notifications page. It displays a table of events with the following data:

ID	Name	Code	Color	Type	Notifications
4	air_pollue	air_pollue		Info	Enabled

The page includes a search bar for events and a description: "Events are used to track, log, and work with important events that happen on the device. [Learn more in documentation](#)". The left sidebar and top bar are identical to the dashboard view.

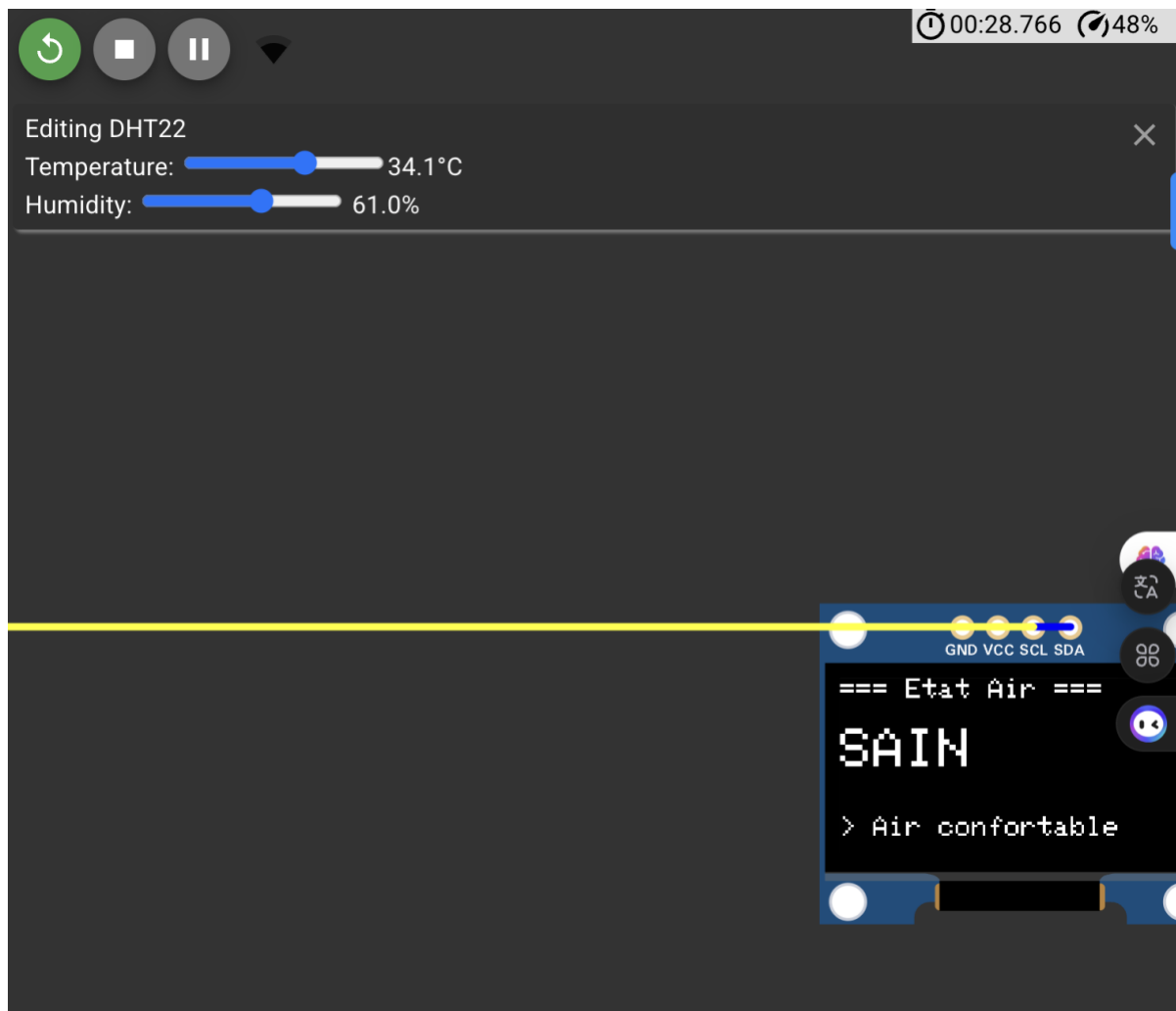
8. Validation et tableau de tests

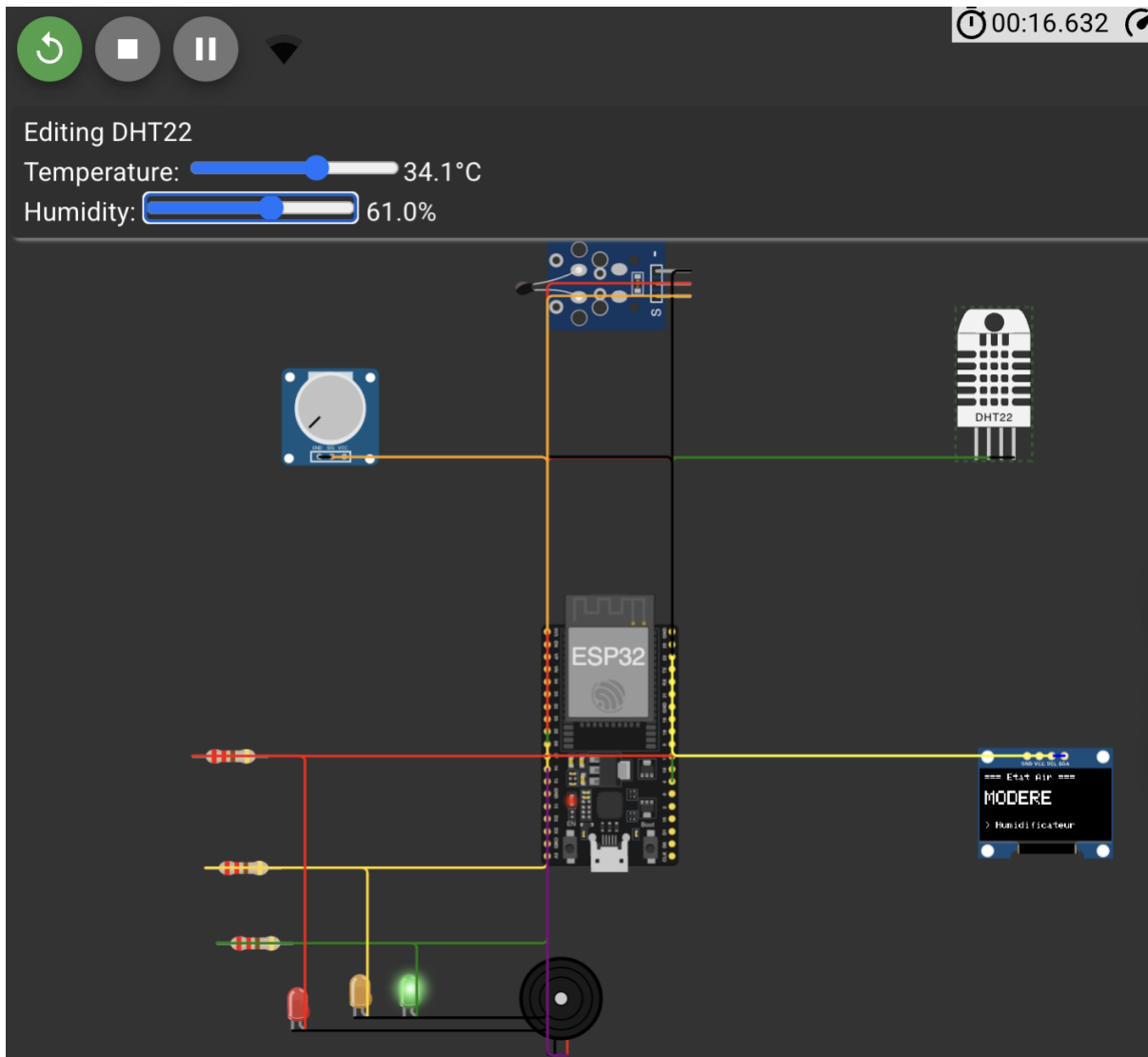
Trois profils ont été testés en faisant varier les capteurs (DHT22 et potentiomètre) afin de couvrir les trois états.

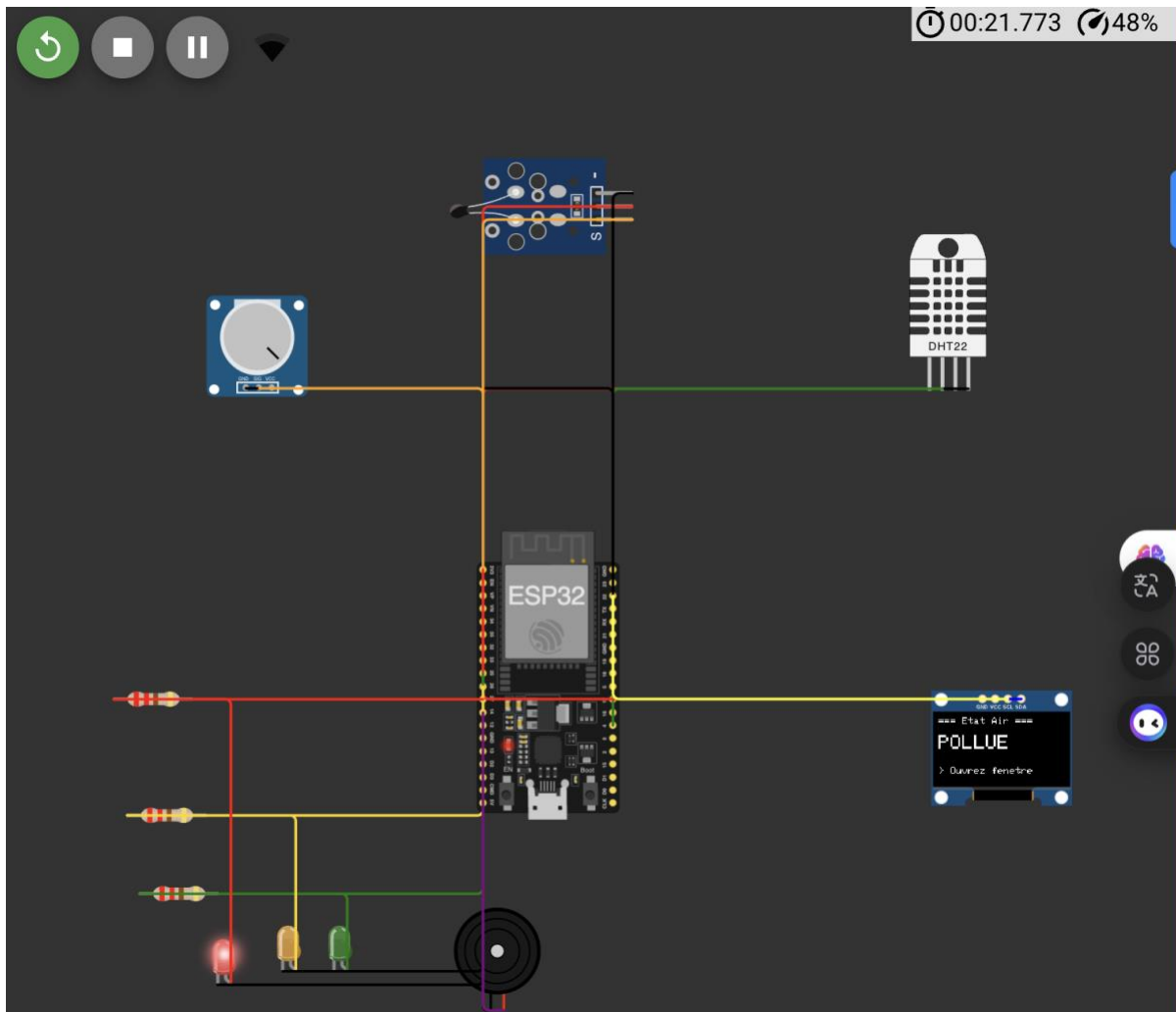
Profil	Entrées (T / HR / CO2)	Indice attendu	État attendu	Résultat obtenu
Air sain	22 C / 50 % / 10 %	≥ 60	SAIN (LED verte)	Conforme
Air modéré	27 C / 35 % / 50 %	30 - 59	MODÉRÉ (LED orange)	Conforme
Air pollué	30 C / 15 % / 90 %	< 30	POLLUÉ (LED rouge + buzzer)	Conforme
DHT en défaut	NaN	inchangé	dernière valeur conservée	Conforme

Vérifications complémentaires :

- Cohérence de l'indice global calculé avec les valeurs des capteurs : OK.
- Alternance des deux écrans OLED toutes les 3 s et lisibilité : OK.
- Le buzzer ne sonne qu'en état critique et s'arrête au retour à la normale : OK.







9. Extensions réalisées

- Conseil contextuel sur l'OLED : "Ouvrez fenêtre" si CO2 élevé, "Humidificateur" si HR < 30 %.
- Prise en compte du temps passé dans chaque état de qualité d'air (tableau stateSeconds).
- Utilisation du capteur NTC comme température secondaire (moyenne avec le DHT22).

9.1 Vers une version réelle

Le potentiomètre simulant le CO2 pourrait être remplacé par un véritable capteur CCS811 (CO2/COV en I2C) ou MQ-135, avec un étage de conditionnement et une phase de calibration. Le reste de l'architecture logicielle resterait identique.

10. Difficultés rencontrées

- Conversion de la lecture analogique du NTC en degrés Celsius via l'équation Beta.
- Calibration des seuils de l'indice pour des états bien distincts.
- Suppression du clignotement de l'OLED (redessin uniquement au changement d'écran).
- Connexion Blynk via Wokwi-GUEST et synchronisation des datastreams avec le dashboard.

11. Conclusion

Le système répond à l'ensemble des exigences du cahier des charges : acquisition multi-capteurs, calcul d'un indice de confort pondéré, classification en trois niveaux, affichage local multi-écrans, signalisation LED/buzzer et supervision à distance via Blynk avec notifications. Le code est structuré en fonctions, commenté et non bloquant (millis()). Le projet constitue une base solide, directement transposable à une version matérielle réelle avec de vrais capteurs de qualité d'air.

12. Annexes - Liens

Simulation Wokwi : wokwi.com/projects/465825450297283585

Dépôt Git : github.com/mohamed-elhaimer/smarthome-air-quality-esp32